Politechnika Wrocławska

Programowanie obiektowe Lab 1

Dr inż. Paweł Maślak

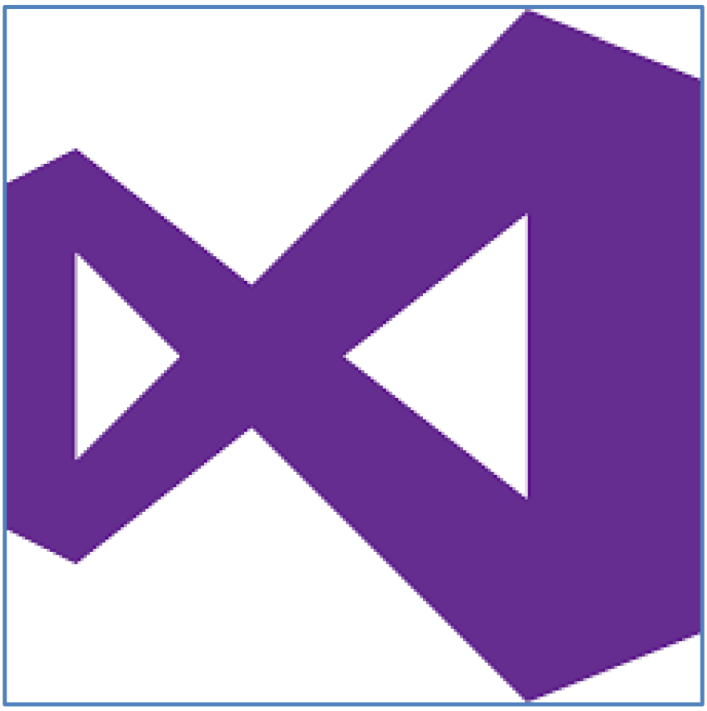
pawel.maslak@pwr.edu.pl

Laboratorium 1: Środowiska pracy – pierwsze przykłady

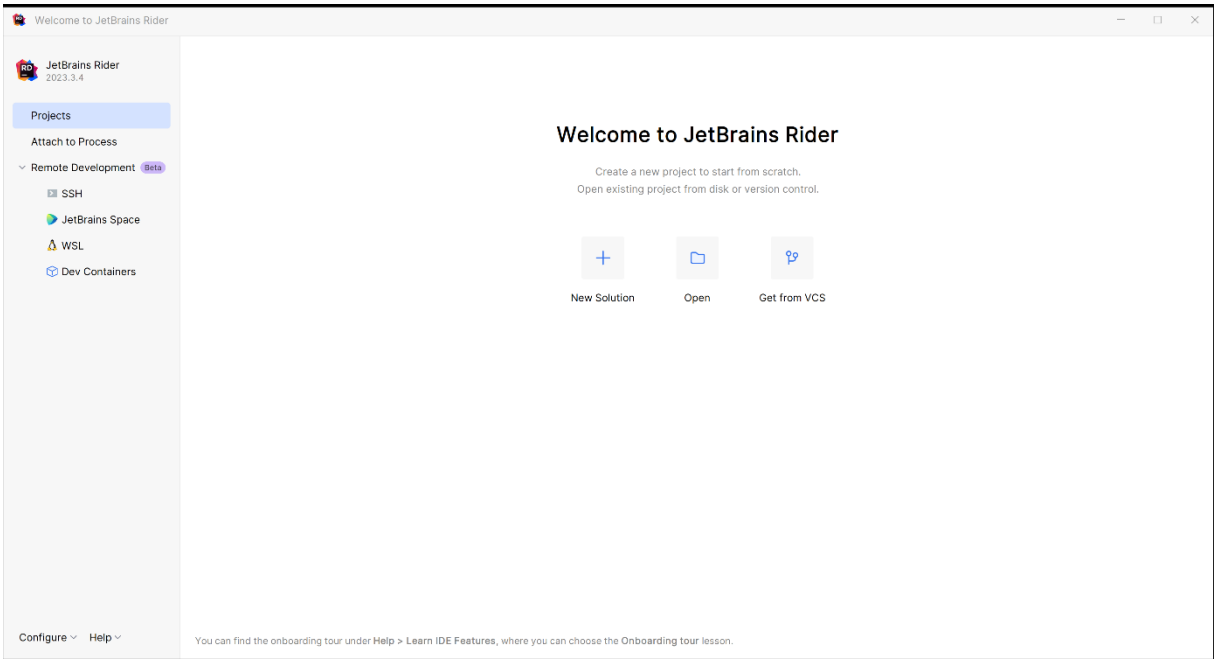
1. Oprogramowanie do zajęć to:

- Rider: The Cross-Platform .NET IDE from JetBrains
  https://www.jetbrains.com/rider/
  program dla dydaktyki jest darmowy, odnowienie za 11 miesięcy
  Free Educational Licenses - Community Support (jetbrains.com)

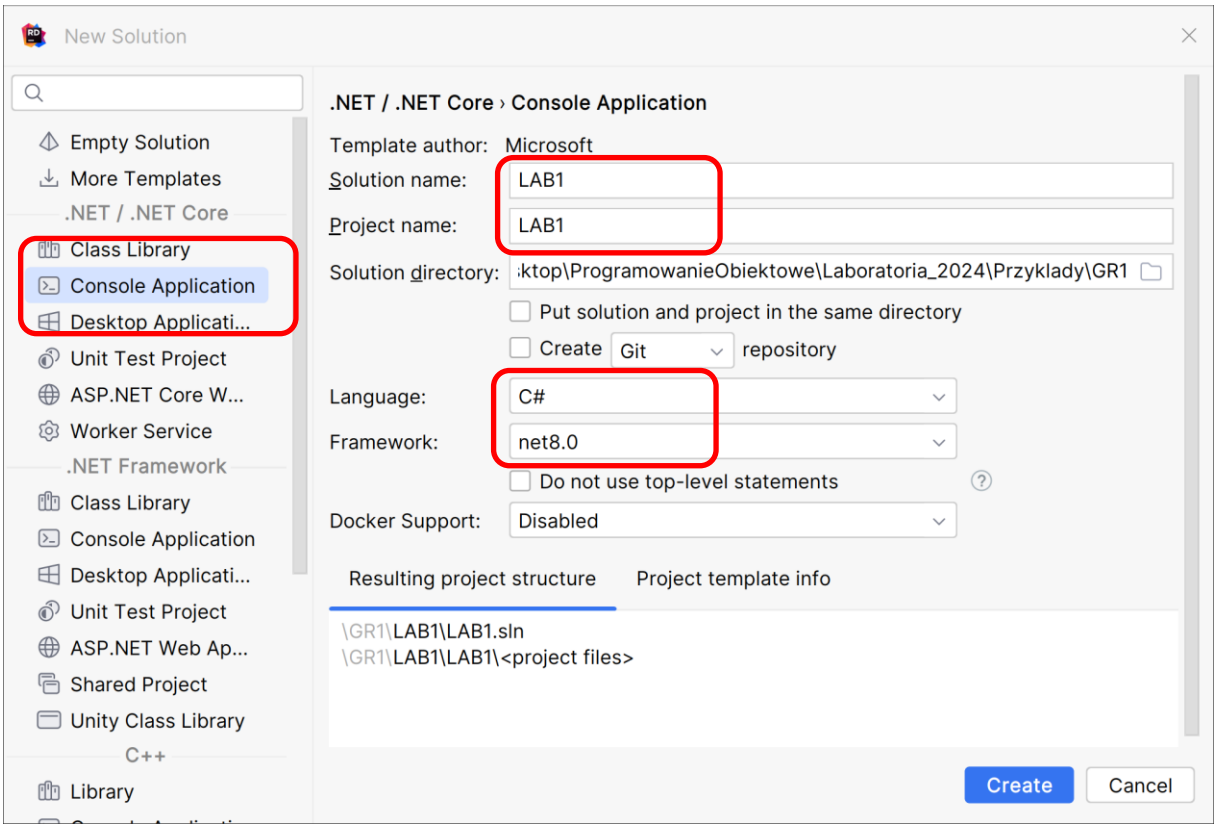
- Visual studio
  wersja community 2022 do pobrania - darmowe dla edukacji
  Visual Studio: środowisko IDE i edytor kodu dla deweloperów i zespołów (microsoft.com)
  https://visualstudio.microsoft.com/pl/



2. Po zainstalowaniu uruchamiany Program Rider:

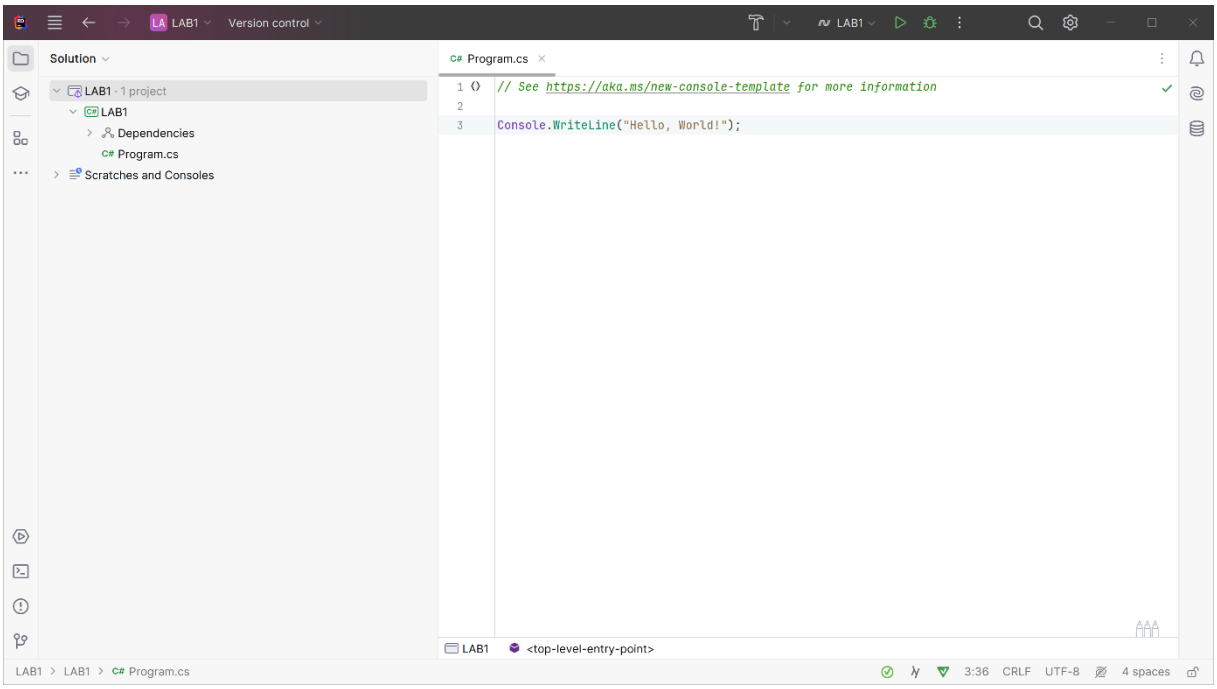


3. Utworzymy **pierwszy program**. Uruchamiamy „New Solution” z następującymi parametrami:

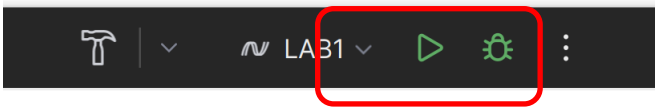


Pojawi się pierwszy program słynne „Hello world”

```
Console.WriteLine("Hello, World!");
```



Program kompilujemy:



I uzyskujemy wynik:

Hello, World!

Process finished with exit code 0.

Pierwszy program został wygenerowany automatycznie, lecz daleko mu do programowania obiektowego.

Tylko tym razem zaprezentowane zostanie jak to wszystko powinno wyglądać, a przy następnych ćwiczeniach, należy pamiętać, że kod, który widnieje w przykładach powinien znaleźć się w funkcji Main() (oczywiście będą od tego wyjątki). Program „Hello World!" wygląda, w szkielecie obiektowym w następujący sposób:

```
using System;
public class FirstProgram
{
    public static void Main(string[] args)
    {
        Console.WriteLine("Hello World!");
    }
}
```

Warto zauważyć, że funkcja Main() znajduje się w klasie FirstProgram. Należy pamiętać, że klasa powinna mieć taką samą nazwę jak nazwa pliku z kodem źródłowym w języku C#. Nawiasy klamrowe „{" oraz „}", oznaczają początek i koniec bloku kodu. Interpretuj ich znaczenie jako rozpocznij i zakończ.

Wspomniana już wcześniej dyrektywa using, mówi kompilatorowi, że przed utworzeniem pliku wykonywalnego (w systemie Window jest to plik z rozszerzeniem .exe) dołączył do tego konkretnego programu, kod biblioteki System. Oczywiście w przyszłości będziesz dołączał coraz to więcej bibliotek, dzięki temu uzyskujesz możliwość bardziej efektywnego programowania.

Powyższy szablon, będziesz wykorzystywał praktycznie zawsze, oczywiście na coraz to bardziej zaawansowanych etapach programowania, będziesz dołączał coraz więcej bibliotek i pisał coraz więcej linii kodu. Zapewne zauważyłeś, że prawie każdy wiersz kończy się średnikiem, który jest ważnym elementem programowania w języku C#. Mówi on kompilatorowi, że dana instrukcja została zakończona. Pominięcie średnika to jeden z najczęstszych błędów popełnianych przez programistów. Jeżeli program z jakiegoś powodu nie działa, lepiej upewnić się, czy wszędzie, gdzie należy został umieszczony średnik.

Na pewno zastanawiasz się dlaczego nie ma średnika po public class FirstProgram oraz public static void Main(string[] args). Pierwsza nazwa, to nazwa klasy a ściślej klasy publicznej, ale to temat na programowanie obiektowe, natomiast druga to publiczna statyczna funkcja.

Po nazwie funkcji nie można stawiać średnika. Funkcję i klasę można rozumieć na razie jako nazwę bloku kodu przed nawiasami klamrowymi. Jak już zostały „wywołane" funkcje, to warto wspomnieć, że wartość, którą podajesz w nawiasach okrągłych po nazwie tej funkcji, w programowaniu nazywamy argumentem.

Wracając do funkcji wypisujących, to w języku C# funkcja Console.WriteLine() nie jest jedyną funkcją, której zadaniem jest wypisanie czegoś w terminalu. Oprócz niej, C# posiada jeszcze funkcję Console.Write(), która wykonuje tę samą operację wypisu, ale w trochę inny sposób.

```
using System;
public class FirstProgram
{
    public static void Main(string[] args) {
        Console.Write("Hello");
        Console.Write("World!");
    }
}
```

Można zauważyć, że pomimo użycia dwóch funkcji wypisujących, podane dwa napisy złączyły się w jeden i wypisały się w jednej linii w terminalu. Dzieje się tak ponieważ funkcja Console.WriteLine() domyślnie na końcu napisu, dodaje znak przejścia do następnej linii - „\n”, a funkcja Console.Write() tego nie robi. Na powyższych rysunkach widnieje napis „Process finished with exit code 0”, oznacza on, że napisany przez programistę kod został wykonany bez żadnego błędu. Na tym etapie spróbuj trochę poeksperymentować z funkcjami Console.WriteLine() i Console.Write(). Spróbuj wypisać różne teksty albo wiele wierszy tekstu i sprawdź, jak będzie zachowywał się twój program.

Zadania praktyczne:

1. Napisz program, który wyświetli Twoje imię.
2. Napisz program, który wyświetli na ekranie wiele wierszy tekstu, z których każdy będzie zawierać imię jednego z twoich znajomych.

4. Podstawowe operacje, komentarze

Funkcje **Console.WriteLine()** i **Console.Write()** języka C# nie tylko służą do wypisania stringów – łańcuchów tekstowych. Można również wypisywać nimi liczby, pojedyncze znaki, wartości logiczne (true lub false), zmienne oraz wiele innych struktur, które zostaną później omówione. C# obsługuje szereg operacji arytmetycznych, bez potrzeby dołączania do programu zaawansowanych bibliotek. Zestaw takich podstawowych operacji arytmetycznych znajduje się na poniższym listingu kodu.

```
using System;
public class FirstProgram
{
    public static void Main(string[] args) {
        Console.WriteLine(2 + 2); //dodawanie
        Console.WriteLine(3 - 2); //odejmowanie
        Console.WriteLine(3 * 3); //mnożenie
        Console.WriteLine(12 / 4); //dzielenie
        Console.WriteLine(19 % 4); //reszta z dzielenia

    }
}
```

Jako wynik uzyskamy:

```
4
1
9
3
3

Process finished with exit code 0.
```

W przypadku działań arytmetycznych, podanych jako argument do funkcji Console.WriteLine(), nie używa się cudzysłowów. Generalnie cudzysłowie używa się tylko przy stringach, natomiast przy innych argumentach, raczej nie wprowadza się cudzysłowu, wyjątek będzie stanowił pojedynczy znak, czyli char, w tym przypadku obowiązuje pojedynczy cudzysłów (‘’). Listing poniżej to dobry przykład, że języka programowania (i uwaga nie tylko C#), da się używać jako pełnoprawnego kalkulatora.

Każdy język programowania bardzo dobrze poradzi sobie z każdym równaniem i nie trzeba się bać o kolejność wykonywania działań, C# jak każdy inny język programowania, bez problemu obliczy równanie 2 + 2 × 3. Dobrze wiadomo, że jego wynik powinien wynosić 8, bo przecież 2 pomnożone przez 3 daje 6, a najpierw wykonuje się mnożenie, a następnie do wyniku mnożenia dodaje 2. Ale warto sprawdzić, jak poradzi sobie z tym język C#.

```
using System;
public class FirstProgram
{
    public static void Main(string[] args)
    {
        Console.Write(2+2*3); //powinno wypisać się 8
    }
}
```

Jako wynik uzyskamy:

```
8
Process finished with exit code 0.
```

Zapewne zastanawiasz się, co robią „//” w C#, bo w obydwu poprzednich listingach, pojawiły się takie znaki. Oznaczają one początek komentarzy. Komentarz nie jest niczym obowiązkowym, ale często ułatwia odnalezienie się w kodzie innego programisty lub gdy będziesz musiał wrócić do niego po jakimś czasie. Istnieje też możliwość dodania komentarza blokowego, tzn. obejmującego kilka wierszy w kodzie. Takim komentarzem, można uniemożliwić wykonanie się danego bloku kodu, na przykład tak jak w poniższym listingu:

```
using System;
public class FirstProgram
{
    public static void Main(string[] args)
    {
        /*
        Console.WriteLine("To się nie wypisze");
        */
    }
}
```

Jako wynik uzyskamy:

```
Process finished with exit code 0.
```

Gdy kompilator natrafi na znak rozpoczęcia komentarza, pomija zawarty po nim tekst, przechodząc do kolejnej linii lub w przypadku komentarza blokowego do linii, w której następuje znak końca komentarza.

Niektóre środowiska programistyczne zmieniają kolor fragmentu z komentarzem, co można zauważyć na powyższych listingach, gdzie został on zmieniony na zielony. To podpowiada programiście by mógł on łatwiej zauważyć, że kod nie zostanie wykonany. Podczas nauki programowania, przydatna będzie możliwość „skomentowania" niektórych fragmentów kodu w celu sprawdzenia, w jaki sposób wpłynie to na wynik działania programu. Te „zakomentowanie" będzie polegało na umieszczeniu znaków komentarza przy wierszach, których mają się nie kompilować.

Jeśli skomentowany zostanie kod, który jest wymagany, czyli na przykład dyrektywa *using System* lub jakaś inna, odnosząca się do biblioteki, która będzie używana w kodzie, program może nie wykonać się poprawnie. Na końcu komentarza nie stawia się średnika, ze względu na to, że komentarz i tak jest ignorowany przez kompilator, więc nie ma to sensu. Oprócz operacji arytmetycznych, które można wykonać na liczbach istnieje też operacja połączenia dwóch stringów. Operację taką w programowaniu nazywamy konkatenacją i wykonujemy ją za pomocą znaku dodawania „+". Jak wygląda to w praktyce? Powiedzmy, że chcesz wypisać na ekranie konsoli napis „Cześć! Lubisz programować w C#?", ale używając połączenia stringów „Cześć!" i „Lubisz programować w C#?". Operacja konkatenacji stringów w języku C#, wygląda w następujący sposób:

```
using System;
public class FirstProgram
{
    public static void Main(string[] args)
    {
        Console.Write("Cześć!" + "Lubisz programować w C#?");
    }
}
```

Jako wynik uzyskamy:

```
Cześć!Lubisz programować w C#?
Process finished with exit code 0.
```

Operacja się wykonała, ale tekst się scalił w jeden, bez spacji. Oczywiście można dodać trzeci string, będący pustą spacją i kod wykona się poprawnie, gdyż operacja konkatenacji nie ogranicza liczby stringów, które da się złączyć w jeden a ich ilość może być dowolna, dlatego taki kod, jak na poniższym listingu, jest jak najbardziej poprawny:

```
using System;
public class FirstProgram
{
    public static void Main(string[] args){
        Console.Write("Cześć" + "!" + "Lubisz programować w C#?");
    }
}
```

Jako wynik uzyskamy:

```
Cześć!Lubisz programować w C#?
Process finished with exit code 0.
```

Zadania praktyczne:

1. Napisz program, który wyświetli wynik działania:

   $100 - 5 \times (2 + 3) - 5\ 2$.

2. Napisz program, który wyświetli Twoje imię i nazwisko, Twojego kolegi oraz Prowadzącego będące osobnymi stringami.

5. Zmienne i stałe, podstawowe typy danych, konwersja typu

Wiesz już, jak napisać prosty program komputerowy. Teraz czas, by zaimplementować pierwszy program, który będzie się komunikował z użytkownikiem. Można spróbować napisać program, którego zadaniem będzie wyświetlenie tekstu, który poda użytkownik. Język C# posiada funkcję, której zadaniem jest pobranie od użytkownika, informacji potrzebnych w wykonaniu jakiegoś programu.

Funkcja ta, to **Console.ReadLine().** Niestety, aby napisać taki program, nie tylko potrzebujesz znać tę funkcje, ale przede wszystkim potrzebujesz utworzyć zmienną, czyli taką konstrukcję programistyczną, która posiada swoją nazwę i wartość. Nazwa zmiennej powinna mieć od 1 do 40 znaków, zawsze musi się zaczynać od litery oraz zawierać tylko litery, liczby i znaki podkreślenia („_”), a co najważniejsze jej nazwa nie może być słowem kluczowym w danym języku programowania.

W większości języków programowania, zmienne powinny mieć ustalony typ danych, które przechowują. Język programowania, który wymaga podania typu danych przed nazwą zmiennej, nazywamy językiem silnie typowanym. Istnieją również języki dynamicznie typowane, które nie wymagają podania typu danych, a język programowania sam rozpoznaje wartość zmiennej i na jej podstawie określany jest jej typ. Język C# jest językiem silnie typowanym, co oznacza, że typy zmiennych są sprawdzane podczas kompilacji, a nie podczas działania programu.

Pomimo, że wprowadzono w nim słowo kluczowe var (mogłeś się z nim spotkać programując w języku JavaScript), które pozwala na utworzenie zmiennej bez podania jej typu to jednak kompilator nadal musi ustalić typ zmiennej w trakcie kompilacji. W większości przypadków kompilator domyśla się, jaki typ danych ma mieć zmienna, jednak czasem można zauważyć pewne efekty uboczne w czytelności kodu. Aby więc utworzyć zmienną, trzeba wymyśleć dla niej jakąś nazwę i przypisać do niej wartość, za pomocą operatora przypisania, którym w języku C# jest znak ‘=’ (oczywiście nazwa zmiennej musi być poprzedzona słowem kluczowym **var**). W poniższym przykładzie utworzona została zmienna o nazwie **myVar**, która będzie przechowywać wartość 1, a następnie jej wartość zostanie wypisana w konsoli.

```
public class FirstProgram
{
    public static void Main(string[] args)
    {
        var myVar = 1;
        Console.Write(myVar);
    }
}
```

Jako wynik uzyskamy:

```
1
Process finished with exit code 0.
```

Jak już wiesz, kompilator sam musi rozpoznać typ zmiennej, gdy tworzysz ją używając słowa kluczowego var. Dlatego w poniższym przykładzie utworzona została zmienna, która będzie przechowywała łańcuch tekstowy.

```
using System;
public class FirstProgram
{
    public static void Main(string[] args)
    {
        var my2NdVar = "Hi!";
        Console.Write(my2NdVar);
    }
}
```

Jako wynik uzyskamy:

```
Hi!
Process finished with exit code 0.
```

Drugi sposób tworzenia zmiennych wymaga podania typu zmiennej przed jej nazwą. Poznałeś już dwa typy danych – string, który służy do przechowywania ciągu znaków, oraz int, który służy do przechowywania liczb całkowitych. W programowaniu, nie będziesz miał do czynienia tylko z tymi typami danych. W języku C#, istnieje o wiele więcej podstawowych typów danych. Oprócz typów **int**, **string** istnieje typ **bool** (przyjmujący wartości logiczne true i false) oraz **float** (na końcu wartości typu float należy dopisać ‘f’ np. 3.14f) ale również:

- **byte**, który może przechowywać liczbę w postaci 8-bitowej (0 – 255)
- **char**, który przechowuje pojedynczy znak
- **decimal**, który przechowuje wartość dziesiętną z dokładnością do 28/29 cyfr
- **long**, który przechowuje 64-bitowe liczby całkowite
- **sbyte**, który może przechowywać liczbę w postaci 8-bitowej (-127 - 127)
- **short**, który może przechowywać liczby całkowite 16-bitowe
- **uint**, który przechowuje liczby całkowite bez możliwości przechowywania liczb ujemnych
- **ulong**, który może przechowywać 64-bitowe liczby całkowite bez możliwości przechowywania liczb ujemnych
- **ushort**, który przechowuje liczby całkowite 16-bitowe bez możliwości przechowywania liczb ujemnych

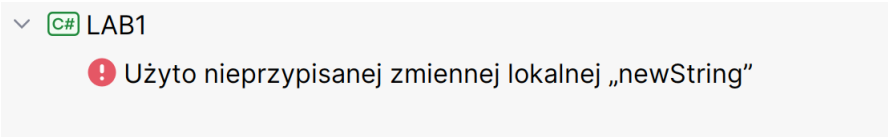
Wszystkie te typy, mają przedziały, w których działają (i nie można tego przedziału przekroczyć). W poniższym przykładzie utworzona została zmienna typu całkowitego przechowującą wartość 1, zmienna typu string o nazwie newString i zmienną typu char przechowującą pojedynczy znak „z”.

```
using System;
public class FirstProgram
{
    public static void Main(string[] args)
    {
        int number = 1;
        String newString;
        char newChar = 'z';
    }
}
```

Jak widać zmienna newString nie posiada żadnej przypisanej jej wartości, ale istnieje ona w pamięci programu. Takie rozwiązanie nazywamy deklaracją zmiennej. Polega ono na tym, że tworzymy zmienną, która w przyszłości będzie przechowywać jakąś wartość, którą ustalisz w późniejszym etapie działania programu. Można spróbować przetestować, co wypisze program, gdy odwołasz się do zmiennej, która nie przechowuje w tej chwili żadnej wartości, a następnie wypiszemy zmienną uprzednio podając jej wartość.

```
using System;
public class FirstProgram
{
    public static void Main(string[] args)
    {
        int number = 1;
        String newString;
        Console.Write(newString);
        char newChar = 'z';
    }
}
```

Jako wynik uzyskamy:



IDE wykryło błąd i podkreśliło go czerwonym kolorem, a także zablokowało przycisk „Run” co uniemożliwia kompilację. Po kliknięciu w czerwoną żarówkę, która w środowisku developerskim odpowiada za podpowiadanie co należy zmienić, aby pozbyć się błędu, pojawiła się sugestia, że należy zainicjować zmienną newString. Operacja inicjalizacji polega na dodaniu wartości do zadeklarowanej zmiennej, co zaprezentowano na poniższym listingu:

```
using System;
public class FirstProgram
{
    public static void Main(string[] args)
    {
        String new_string;
        new_string = "Hi! What's up?";
        Console.WriteLine(new_string);
    }
}
```

Zapewne zauważyłeś, że utworzenie nowej zmiennej oraz jej deklaracja i inicjacja wymaga dodania średnika na końcu instrukcji. Dodatkowo, na zmiennych można wykonywać operacje arytmetyczne. Jeśli zmienne są typu string, bez problemu można wykonać na nich konkatenację stringów. Na poniższym przykładzie znajduje się program, który dodaje liczby a i b, zapisując je do zmiennej c, a następnie wypisuje wartość zmiennej c do konsoli.

```
using System;
public class FirstProgram
{
    public static void Main(string[] args)
    {
        int a = 10, b = 5;
        var c = a + b;
        Console.WriteLine(c);
    }
}
```

Jeśli istnieje potrzeba utworzenia kilku zmiennych tego samego typu, można posłużyć się tym sposobem, który został przedstawiony na powyższym listingu, czyli podać typ zmiennych, które będą utworzone, a następnie ich nazwy i przypisać im wartości, które będą posiadać.

Pamiętaj, że każdą nową zmienną, należy oddzielić przecinkiem. Oczywiście na końcu takiej instrukcji, musi znaleźć się średnik. Przykładowo da się utworzyć trzy zmienne logiczne, które będą przechowywały kolejno wartości true, false oraz true w następujący sposób:

```
using System;
public class FirstProgram
{
    public static void Main(string[] args)
    {
        bool a = true, b = false, c = true;
    }
}
```

Niekiedy zdarza się, że programista potrzebuje zmienić typ z jednego, na inny. Operację zmiany typu danych nazywa się konwersją typu lub bardziej popularnie – rzutowaniem. Oczywiście nie ma możliwości by rzutować każdy typ danych, na dowolnie inny, bo niektórych operacje konwersji nie da się wykonać już w IDE, które po prostu podkreśli błąd.

Najprostszym sposobem, by wykonać rzutowanie, jest umieszczenie w nawiasach okrągłych „()”, nazwę typu, który chce się uzyskać. Nawiasy zawierające nazwę typu, powinny znaleźć się przed wartością, jeśli chcesz dokonać rzutowania bezpośrednio wartości wpisanej „z palca” lub przed nazwą zmiennej – jeśli dokonywane jest rzutowanie wartości zmiennej pośrednio.

Przykładowo, na poniższym listingu stworzona została zmienna o nazwie pi, która jest typu int, a przypisano jej wynik operacji rzutowania liczby 3,14f, na typ int. W drugim przypadku, dla utworzonej zmiennej typu float o nazwie e i przypisano wartość 2,7 (tyle wynosi tak zwana stała Eulera, będąca podstawą logarytmu naturalnego). Następnie utworzone zostały zmienne eInt, typu int, która przechowuje wartość tej zmiennej po rzutowaniu na ten typ. Cały kod prezentuje się tak:

```
using System;
public class FirstProgram
{
    public static void Main(string[] args)
    {
        int pi = (int) 3.14f;
        float e = 2.7f;
        int eInt = (int) e;
        Console.WriteLine(pi);
        Console.WriteLine(eInt);
    }
}
```

Jako wynik uzyskamy:

```
3
2

Process finished with exit code 0.
```

Może wydawać się, że liczba 2,7 jest bliżej liczby 3, to czemu wartość zmiennej eInt nie jest równa 3? Przecież taką dużą część dziesiętną liczby powinno zaokrąglić się do góry. Faktycznie tak jest, ale rzutowanie to nie operacja zaokrąglenia liczby, tylko zmiany typu. W przypadku konwersji typu float na int, działa to tak, że odcina to co jest po przecinku, pozostawiając wartość, która występuje tylko przed przecinkiem. Dla rzutowania typu int na float, do wartości typu całkowitego dopisywane jest 0 po przecinku. Problem istnieje podczas konwersji typu int na string. Takie rzutowanie jest wykonalne, ale nie można go wykonać za pomocą podania typu – w tym przypadku string – w nawiasach okrągłych. Operacja ta, odbywa się za pomocą funkcji ToString(). Znamy już kilka funkcji, jednak w przypadku tej nie będziesz ją wywoływał z argumentami, tylko po kropce przy nazwie zmiennej. Przykład rzutowania z typu int na string może wyglądać chociażby tak:

```
using System;
public class FirstProgram
{
    public static void Main(string[] args)
    {
        int i = 1;
        string j = i.ToString();
    }
}
```

Dodatkowym sposobem zmiany typu, jest użycie funkcji **Convert.ToInt32(),** gdzie jako parametr podajemy wyrażenie dowolnego typu. Convert nie tylko przydaje się do konwersji na int, możliwa jest również konwersja na typ bool za pomocą **Convert.ToBoolean(),** double za pomocą **Convert.ToDouble()** a nawet na datę za pomocą **Convert.ToDateTime().** Zamiana typu int na bool (ale też każdego typu liczbowego na bool) za pomocą funkcji **Convert.ToBoolean(),** daje false, gdy wartość jest równa 0, a true gdy wartość jest różna od 0. Dla wartości 24441395 i -24441395, po konwersji na bool zmienna będzie przechowywała true, a false tylko dla wartości 0. Przykład rzutowania z użyciem funkcji Convert znajduje się poniżej.

```
using System;
public class FirstProgram
{
    public static void Main(string[] args)
    {
        string i = "122022";
        int j = Convert.ToInt32(i);
    }
}
```

Istnieje też operacja rzutowania typu int na char, zobaczmy to na przykładzie:

```
public class FirstProgram
{
    public static void Main(string[] args)
    {
        int i = 65;
        char z = 'Z';

        //Konwersja
        char j = Convert.ToChar(i);
        int a = Convert.ToInt32(z);
        Console.WriteLine(j);
        Console.WriteLine(a);
    }
}
```

Jako wynik uzyskamy:

```
A
90

Process finished with exit code 0.
```

Myśląc logicznie raczej spodziewalibyśmy się w obu przypadkach błędów, przecież 65 to nie jest pojedynczy znak – to już prędzej string, czyli łańcuch znaków złożony ze znaku ‘6’ i ‘5’, a rzutowanie ‘Z’ na int to w ogóle powinno być błędne, bo przecież logicznie nie ma to sensu zamienić ‘Z’ na cyfrę.

Jednak istnieje tak zwany kod ASCII, który jest kodem liczbowym przyporządkowanym każdemu znakowi. Wszystkie litery, cyfry, znaki niedrukowalne jak na przykład spacja czy enter są typu char i mają odpowiadające im kody ASCII. Teraz może wydawać się to nie przydatne, ale kody ASCII bardzo często są używane w szyfrowaniu wiadomości. Pełną tabelę kodów ASCII można znaleźć w Internecie.

W skrócie litery A – Z, odpowiadają kodom ASCII 65 – 90, dodatkowo w kodach ASCII ważna jest wielkość liter, więc znaki a – z mają inne kody ASCII (od 97 do 122). Czasami programista potrzebuje utworzyć zmienną, której wartości nie da się zmodyfikować podczas wykonywania programu. Często taka zmienna jest wykorzystywana w szablonach – na przykład wzorcu e-maila czy numeru telefonu, definiując jakie znaki są możliwe do wpisania w polu formularza, a zarazem uniemożliwiając wpisać inne znaki w takich polach.

Taką zmienną, której wartości nie da się zmodyfikować nazywamy stałą. Jej wartość określana jest tylko raz, ale można się do niej odwołać w programie wielokrotnie. Stosowanie stałej i odwoływanie się do niej, zamiast wielokrotnego używania takich samych wyrażeń, nie tylko ułatwia konserwację kodu, ale dostarcza dla niej nazwę opisową, zwiększając czytelność.

Wartość stałej w zależności od języka programowania, może być znana na etapie kompilacji i nie może się zmienić w trakcie działania programu lub ustawiona jednorazowo, a potem jej wartość nie może się zmienić. Umownie, nazwy stałych powinny być pisane w całości z dużych liter, a wyrazy je tworzące oddzielone podłogą ('_'). W języku C#, stałą tworzy się za pomocą słowa kluczowego const, a następnie należy podać jej typ i nazwę oraz operatorem przypisania '=' nadać jej wartość. Tak stworzona stała, jest tak zwaną stałą kompilowaną, a jej wartość jest zawsze zawarta w bibliotece w trakcie kompilacji. W C# istnieje również inne słowo kluczowe – readonly, które tworzy stałą niekompilowaną, ustalaną w trakcie działania programu. Generalnie programiści nazywają taki typ stałej jako „pole tylko do odczytu", by samą nazwą podkreślić, że nie jest to stała typu kompilowanego – const.

Zadania praktyczne:

1. Utwórz zmienną typu zmiennoprzecinkowego, przypisując jej wartość przybliżenia liczby Pi.

2. Utwórz zmienną typu łańcuchowego, zawierającą Twoje imię i nazwisko.

3. Utwórz zmienną typu całkowitego o nazwie x i przypisz jej wartość 10, a następnie utwórz zmienną typu zmiennoprzecinkowego o nazwie y, której przypiszesz wynik rzutowania zmiennej x na typ zmiennoprzecinkowy.

4. Utwórz zmienną typu zmiennoprzecinkowego o nazwie a i przypisz do niej wartość 10.78493, a następnie utwórz zmienną typu całkowitego o nazwie b, której przypiszesz wynik rzutowania zmiennej a, na całkowity. Jaki wynik otrzymałeś? Jak myślisz dlaczego?